

Lecture 1: Getting Started with Python

Beginner Notes | Python for GenAI

1. Why Python for GenAI

Python is a programming language used for AI, machine learning, data science, automation, APIs, and Generative AI. In this course, Python is a foundation for building AI applications.

Why Python?	Simple explanation
Easy to read	Relatively simple syntax helps beginners focus on problem-solving.
AI ecosystem	Many AI and data tools have strong Python support.
Practical	Useful for data, APIs, automation, and AI applications.
Career relevance	Supports roles such as AI Engineer, GenAI Engineer, ML Engineer, Data Scientist, and Python/Backend Developer.

A useful learning path:

Python → Programming Fundamentals → AI/ML → LLMs → RAG → AI Agents → AI Applications

Python is not the only language used in AI. Other languages are useful in different parts of technology. We are choosing Python because of its strong connection with the AI and data ecosystem.

2. Installing Python

Download Python from **python.org**. Use a current stable version unless your instructor specifies one.

Windows: Run the installer. If an option to add Python to PATH is provided, enable it, then complete the installation.

macOS: Use the official macOS installer. Do not remove or modify the Python installation supplied by macOS.

System	Verification command
Windows	python --version or py --version
macOS	python3 --version

3. Installing VS Code

Visual Studio Code (VS Code) is a code editor used to write, organize, run, and debug Python programs. Download it from **code.visualstudio.com**.

On Windows, the default installation options are normally suitable. On macOS, move the application into Applications. Then install the **Python extension by Microsoft** from the Extensions panel.

4. Your First Python File

Create a folder such as **Python-Course**, open it in VS Code, and create **hello.py**. The **.py** extension identifies a Python source file.

```
print("Hello World!")
```

Save and run the file. If **Hello World!** appears in the terminal, your environment is ready.

5. print()

The **print()** function displays information on the screen. Python normally executes statements from top to bottom.

```
print("Hello")
print(10)
print(10 + 20)
print("10 + 20")
```

10 + 20 performs a calculation. "10 + 20" is text. Multiple values can be printed in one statement:

```
print("Name:", "Rahul", "Age:", 21)
```

6. sep and end

sep controls what is placed between multiple values. **end** controls what `print()` adds after the output.

```
print("Python", "Java", "C++", sep=", ")
print("2026", "09", "05", sep="-")
print("Hello", end=" ")
print("World!")
```

7. Comments

A **comment** is a note in code that Python does not execute. Comments explain code, improve readability, and can temporarily disable code.

```
# This is a comment
print("Hello")
print("Welcome") # This is also a comment
```

8. Escape Sequences

Escape sequences represent special characters inside strings.

Sequence	Meaning	Example
<code>\n</code>	New line	<code>print("Hello\nWorld")</code>
<code>\t</code>	Tab	<code>print("Name\tAge")</code>
<code>\\</code>	Backslash	<code>print("C:\\Python")</code>
<code>\"</code>	Double quote	<code>print("He said \"Hello\"")</code>
<code>'</code>	Single quote	<code>print('It's Python')</code>

9. Debugging Basics

A **bug** is a mistake in a program. **Debugging** means finding and fixing the mistake.

Error	Meaning
Syntax Error	Python cannot understand the structure, for example because a quote or parenthesis is missing.
NameError	Python sees a name that has not been defined.
Indentation Error	Python's indentation is incorrect. It will be covered more later.
Logical Error	The program runs, but the result is not what we intended.

Debugging process: Read the error → check the line number → inspect the line and nearby lines → fix one problem → run again.

10. Quick Cheat Sheet

Concept	Remember
<code>print()</code>	Displays information.
<code>sep</code>	Separator between multiple printed values.
<code>end</code>	Controls what <code>print()</code> adds at the end.
<code>#</code>	Starts a single-line comment.
<code>\n</code>	New line.
<code>\t</code>	Tab.
<code>\\</code>	Backslash.
<code>.py</code>	Common Python source-file extension.